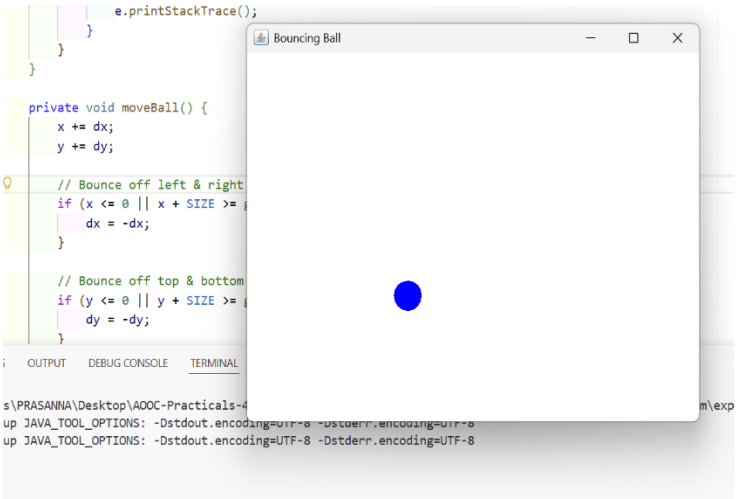
Experiment No. 12

Aim: Design and development of an application demonstrating multithreading.

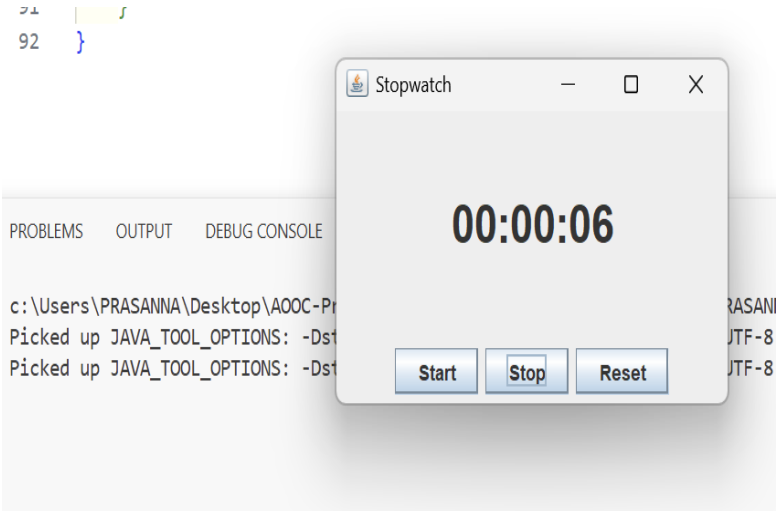
1. Write a program that bounces a blue ball inside a JPanel. The ball should begin moving with a mousePressed event. When the ball hits the edge of the JPanel, it should bounce off the edge and continue in the opposite direction. The ball should be updated using a Runnable.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

public class BouncingBall extends JPanel implements Runnable {

    private int x = 50, y = 50;     // Ball position
    private int dx = 3, dy = 3;     // Direction & speed
    private final int SIZE = 30;    // Ball size
    private boolean running = false;

    private Thread animator;

    public BouncingBall() {
        setBackground(Color.WHITE);

        // Start animation on mouse press
        addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                if (!running) {
                    running = true;
                    animator = new Thread(BouncingBall.this);
                    animator.start();
                }
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
```

```
        // Draw blue ball
        g.setColor(Color.BLUE);
        g.fillOval(x, y, SIZE, SIZE);
    }

    @Override
    public void run() {
        while (running) {
            moveBall();
            repaint();

            try {
                Thread.sleep(10); // control speed
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    private void moveBall() {
x += dx;
        y += dy;

        // Bounce off left & right walls
        if (x <= 0 || x + SIZE >= getWidth())
    {
            dx = -dx;
        }
        // Bounce off top & bottom walls
        if (y <= 0 || y + SIZE >= getHeight())
    {
            dy = -dy;
        }
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame("Bouncing Ball");
        BouncingBall panel = new BouncingBall();
        frame.add(panel);
        frame.setSize(500, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```



2. Create an application of Stopwatch using the concept of Multithreading.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class StopwatchApp extends JFrame implements Runnable {

    private JLabel timeLabel;
    private JButton startBtn, stopBtn, resetBtn;

    private int seconds = 0;
    private boolean running = false;
    private Thread timerThread;

    public StopwatchApp() {
        setTitle("Stopwatch");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        // UI Setup
        timeLabel = new JLabel("00:00:00", SwingConstants.CENTER);
        timeLabel.setFont(new Font("Arial", Font.BOLD, 30));

        startBtn = new JButton("Start");
        stopBtn = new JButton("Stop");
        resetBtn = new JButton("Reset");

        JPanel buttonPanel = new JPanel();
        buttonPanel.add(startBtn);
        buttonPanel.add(stopBtn);
        buttonPanel.add(resetBtn);

        add(timeLabel, BorderLayout.CENTER);
        add(buttonPanel, BorderLayout.SOUTH);

        // Button Actions
        startBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (!running) {
                    running = true;
                    timerThread = new Thread(StopwatchApp.this);
                    timerThread.start();
                }
```

```
        }
    });
    stopBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            running = false;
        }
    });
    resetBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            running = false;
            seconds = 0;
            updateLabel();
        }
    });
}
@Override
public void run() {
    while (running) {
        seconds++;
        updateLabel();
        try {
            Thread.sleep(1000); // 1
second delay
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

private void updateLabel() {
    int hrs = seconds / 3600;
    int mins = (seconds % 3600) / 60;
    int secs = seconds % 60;

    String time = String.format("%02d:%02d:%02d", hrs, mins, secs);
    timeLabel.setText(time);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new StopwatchApp().setVisible(true);
    });
}
}
```